

CRIATURAS & BATALLAS

Juego de Rol de Criaturas por Turnos

Documentación del Proyecto Final
Programación Orientada a Objetos

INTEGRANTES DEL EQUIPO
Any Hayami Cárdenas Ayala — 755446
Patricio Adolfo Villanueva Gio — 744163
Kevin Alejandro Zoloeta Renteria — 755308

1. Descripción General del Proyecto

1.1 Descripción del Proyecto

Criaturas & Batallas es un juego de rol por turnos inspirado en la mecánica de juegos de colección de criaturas. El sistema cuenta con un backend en Java (Javalin) que expone una API REST, una base de datos PostgreSQL para persistencia de sesiones y una interfaz de usuario desarrollada en React + TypeScript con Vite. El jugador elige una criatura inicial y recorre un mapa de zonas enfrentándose a encuentros salvajes y a una secuencia de entrenadores NPC de dificultad creciente.

1.2 Alcance

El proyecto INCLUYE:

- Sistema de combate por turnos con fórmula de daño basada en estadísticas, STAB y ventajas de tipo.
- Siete tipos elementales: FUEGO, AGUA, PLANTA, ELÉCTRICO, TIERRA, HIELO y NORMAL, con tabla de efectividad cruzada.
- Ocho criaturas distintas (Ignis, Aqua, Herba, Voltix, Fumeo, Glacius, Terron, Gorgon) con habilidades pasivas únicas por tipo.
- Sistema de captura con esferas de diferentes tasas de éxito (Normal, HP, Super, Ultra, Tipo, Master).
- Seis entrenadores NPC con dificultad progresiva (Rival Fácil → Rival Normal → Líder Marina → Rival Difícil → Líder Brenno → Campeón Rex).
- Gestión de inventario: pociones, superpociones y esferas de captura.
- Persistencia de sesión en base de datos PostgreSQL (guardar/cargar partida).
- Interfaz gráfica web con pantalla de título, mapa del mundo y pantalla de batalla.

El proyecto NO INCLUYE:

- Multijugador en tiempo real (PvP).
- Tienda o mercado de objetos.
- Más de seis criaturas en el equipo simultáneamente.
- Sistema de evolución de criaturas.
- Guardado automático continuo (el guardado es manual vía API).

1.3 Propósito u Objetivo Principal

El objetivo principal del proyecto es aplicar los principios de la Programación Orientada a Objetos, herencia, encapsulamiento, polimorfismo y abstracción en el contexto de un juego funcional y completo. El proyecto demuestra cómo estructurar un sistema complejo mediante



jerarquías de clases, interfaces, patrones de diseño (factories, controladores REST) y separación de responsabilidades entre capas (modelo, lógica de negocio, API, UI).

1.4 Uso del Sistema

El flujo principal de uso es el siguiente:

- El jugador accede a la interfaz web e ingresa su nombre y elige una criatura inicial (Ignis/Fuego, Aqua/Agua o Herba/Planta).
- Se muestra el mapa del mundo con las opciones: Explorar, Retar al siguiente NPC y Curar equipo.
- Al explorar hay un 40 % de probabilidad de encuentro salvaje; en batalla el jugador puede atacar, usar un objeto, lanzar una esfera o huir.
- Al derrotar a todos los entrenadores NPC, el jugador se convierte en Campeón y termina el juego.
- La sesión puede guardarse y recuperarse desde la base de datos PostgreSQL.

1.5 Usuario Final (Público Objetivo)

El sistema está dirigido a estudiantes y aficionados a los videojuegos de rol que deseen experimentar una mecánica de colección y combate de criaturas. Al mismo tiempo, el proyecto sirve como material educativo para estudiantes de Ingenierías que estudien Programación Orientada a Objetos, ya que el código está extensamente comentado en inglés y es un ejemplo claro de los conceptos fundamentales de la materia.

2. Reflexión sobre el Uso de IA Generativa

2.1 ¿Qué tareas se realizaron con apoyo de IA generativa?

Durante el desarrollo del proyecto se utilizaron herramientas de IA generativa (principalmente GitHub Copilot y Claude) en las siguientes tareas:

- Generación de la fórmula de daño simplificada basada en la Gen-I de Pokémon y su adaptación al contexto del proyecto.
- Creación de los métodos de fábrica estáticos (static factories) en la clase Movimiento para facilitar la creación de movimientos predefinidos.
- Estructuración inicial de los endpoints REST en BattleController y GameController, incluyendo el manejo de los distintos tipos de acción (ATACAR, OBJETO, ESFERA, HUIR).
- Redacción de comentarios JavaDoc y de código explicativo en inglés para las clases principales.
- Generación del esquema SQL (init.sql) para las tablas de la base de datos PostgreSQL.
- Soporte en la depuración de errores de CORS en la configuración de Javalin.

2.2 ¿Cómo ayudó la IA generativa a complementar o mejorar el proyecto?

La IA generativa funcionó como un par de programación virtual que aceleró significativamente las fases de prototipado y documentación. En lugar de invertir tiempo en repetir patrones de código conocidos (getters, setters, constructores, factories), el equipo pudo centrarse en las decisiones de diseño de alto nivel: cómo estructurar la jerarquía de criaturas, cómo separar la lógica de batalla de la sesión de juego, y cómo mapear el modelo de dominio a objetos DTO para la API. Asimismo, la IA propuso mejoras de legibilidad, como el uso de text blocks de Java 15+ para las sentencias SQL multilínea.

2.3 ¿Qué aprendieron al integrar herramientas de Gen AI durante el desarrollo?

- La IA genera código funcional con rapidez, pero requiere revisión crítica: en varias ocasiones propuso soluciones que no seguían el principio de responsabilidad única o introducían acoplamiento innecesario entre clases.
- Es más productivo indicarle a la IA el contexto completo (nombre de clases, interfaz esperada) que pedirle generar código desde cero sin restricciones.
- La IA es especialmente útil para tareas repetitivas y de boilerplate, pero las decisiones arquitectónicas más importantes (como la elección de no extender Entrenador en Batalla) requieren juicio humano.
- Integrar IA en el flujo de trabajo requiere aprender a redactar prompts precisos, una habilidad transferible a otros contextos profesionales y académicos.

3. Diagramas UML

A continuación se presentan los diagramas UML de las principales clases del sistema. Para la versión interactiva y editable de los diagramas.

3.1 Jerarquía de Criaturas

<<abstract>> Criatura	
«abstract» Criatura	
# nombre: String	
# nivel: int	
# hp / hpMax: int	
# ataque / defensa / velocidad: int	
# tipo: TipoElemento	
# movimientos: Movimiento[]	
# tasaCaptura: double	
# bonusDanio / bonusDefensa: double	
+ atacar(mov): int	
+ recibirDanio(d): void	
+ ganarExp(exp): void	
+ subirNivel(): void	
+ curar(cant): void	
+ curarCompleto(): void	
# «abstract» habilidadEspecial(): void	
+ detTasaCaptura(): double	

Subclases de tipo (herencia)	
«abstract» CriaturaFuego extends Criatura	
# calor: int (stacks de calor)	
+ habilidadEspecial(): void // +20% daño c/3 turnos	
«abstract» CriaturaAgua extends Criatura	
+ habilidadEspecial(): void // -15% daño recibido	
«abstract» CriaturaPlanta extends Criatura	
+ habilidadEspecial(): void // regenera HP cada 4 turnos	
«abstract» CriaturaElectrico extends Criatura	
+ habilidadEspecial(): void // +10% velocidad cada 2 turnos	

Relaciones de herencia:

Criatura (abstract) → CriaturaFuego → Ignis, Fumeo
 Criatura (abstract) → CriaturaAgua → Aqua, Glacius, Gorgon
 Criatura (abstract) → CriaturaPlanta → Herba
 Criatura (abstract) → CriaturaElectrico → Voltix
 Criatura implementa: Capturable, Nivelable (interfaces)
 CriaturaNormal → Terron (tipo NORMAL sin subtipo abstracto propio)



3.2 Diagrama de Batalla y Sesión

«abstract» Entrenador
nombre: String
equipo: List<Criatura>
indiceCriaturaActiva: int
+ getCriaturaActiva(): Criatura
+ tieneCriaturasVivas(): boolean
+ cambiarCriatura(idx): void
+ avanzarSiguienteViva(): void
Jugador extends Entrenador
- bolsa: List<Objeto>
- esferas: List<PokeEsfera>
- dinero: int
- medallas: int
+ usarObjeto(idx,c): String
+ lanzarEsfera(idx,c): boolean
+ curarEquipo(): void
EntrenadorNPC extends Entrenador
- dialogo: String
- dificultad: NivelDificultad
- recompensa: int
- derrotado: boolean
+ elegirMovimiento(p,r): Movimiento
+ saludar(): String

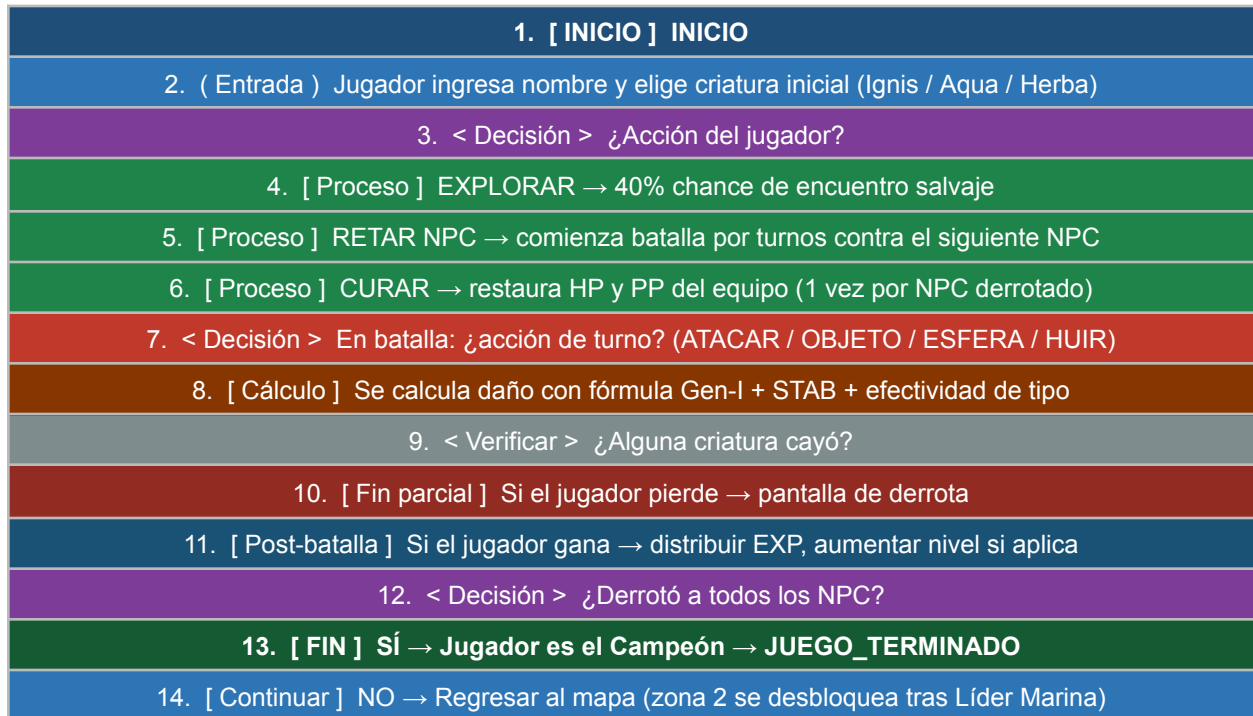
Batalla
- jugador: Jugador
- npc: EntrenadorNPC
- criaturaRival: Criatura
- esEncuentroSalvaje: boolean
- turnoActual: int
- activa: boolean
- ganador: String
+ ejecutarTurno(idx): List<String>
+ calcularDanio(m,a,d): int
+ distribuirExp(log): void
+ haTerminado(): boolean
+ getGanador(): String
+ getRivalActivo(): Criatura

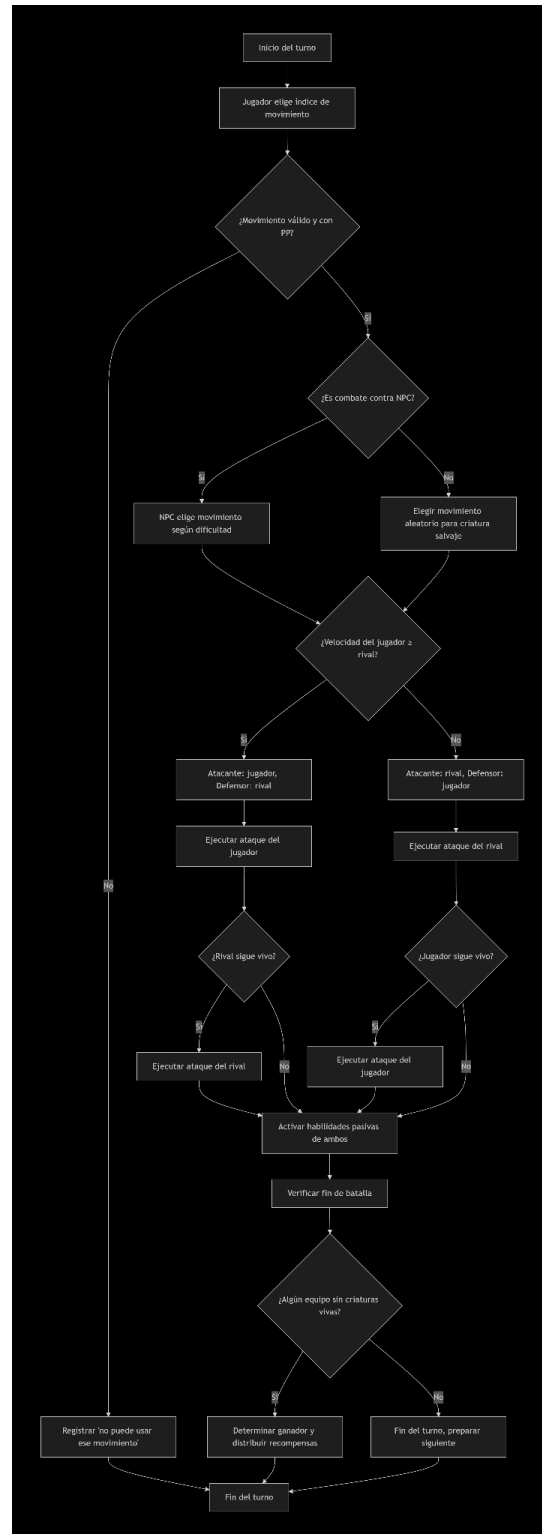
SesionJuego
- jugador: Jugador
- batallaActual: Batalla
- fase: Fase {enum}
- npcs: List<EntrenadorNPC>
- zona: int
- puedesCurar: boolean
+ iniciar(nombre,idx): void
+ explorar(): Batalla
+ retarNpc(): Batalla
+ cerrarBatalla(): void
+ curarEquipo(): void
+ guardarSesion(id): void
+ cargarSesion(id): void



3.3 Diagrama de Flujo — Proceso Principal

El siguiente diagrama describe el flujo principal de una partida completa:





4. Documentación Técnica

A continuación se documenta el propósito, parámetros, valor de retorno y comportamiento de las clases y métodos más importantes del proyecto.

4.1 Clase Criatura (abstract)

Paquete: juego.criaturas | Implementa: Capturable, Nivelable

Clase base abstracta de la que heredan todos los tipos de criaturas. Centraliza la lógica de estadísticas, daño, experiencia, captura y curación.

Método / Campo	Firma	Descripción
Constructor	Criatura(nombre, nivel, baseHp, baseAtk, baseDef, baseVel, tipo, tasaCaptura, movs)	Escala las estadísticas base según el nivel: hpMax = baseHp + (nivel-1)*5, atk += (nivel-1)*3, etc.
atacar()	public int atacar(Movimiento mov)	Consume un PP del movimiento y devuelve su poder base. Si el movimiento no tiene PP retorna 0.
recibirDanio()	public void recibirDanio(int danio)	Reduce hp en danio, con mínimo 0.
ganarExp()	public void ganarExp(int exp)	Acumula experiencia y llama a subirNivel() mientras expTotal >= expSiguienteNivel.
subirNivel()	public void subirNivel()	Incrementa nivel, recalcula umbral de siguiente nivel y aumenta stats. Cura a HP máximo al subir.
detTasaCaptura()	public double detTasaCaptura()	Implementación de Capturable. Retorna tasaCaptura * (1 - hpRatio * 0.7): mayor probabilidad de captura con HP bajo.
habilidadEspecial()	public abstract void habilidadEspecial()	Método abstracto que cada subtipo (Fuego, Agua, Planta, Eléctrico) implementa con su habilidad pasiva única.
curarCompleto()	public void curarCompleto()	Restaura hp = hpMax y llama restaurarPP() en todos los movimientos no nulos. Usada en el Centro de Curación.



4.2 Clase Batalla

Paquete: juego.batalla

Gestiona una sesión de combate completa entre el jugador y un NPC o criatura salvaje. No extiende Entrenador (una batalla no ES un entrenador). Mantiene el estado del turno, el registro de eventos y declara al ganador.

Método	Firma	Descripción
ejecutarTurno()	public List<String> ejecutarTurno(int movIdx)	Ejecuta un turno completo: determina orden por velocidad, aplica ataques, ejecuta habilidades pasivas y verifica fin de batalla. Retorna log de eventos.
calcularDanio()	public int calcularDanio(Movimiento, Criatura atk, Criatura def)	Fórmula Gen-I simplificada: base = $(2L/5+2)*poder*(atk/def)/50+2$. Aplica STAB (+50%), efectividad de tipo, bonuses pasivos y varianza aleatoria (85%-100%).
verificarFin()	private void verificarFin(List<String>)	Detecta si algún bando quedó sin criaturas vivas. Si el NPC tiene más criaturas, cambia automáticamente. Si el jugador gana, distribuye EXP y otorga recompensa.
distribuirExp()	public void distribuirExp(List<String>)	Calcula expBase = nivel_rival * 50 y lo divide entre todas las criaturas vivas del equipo. Registra subidas de nivel en el log.
Constructor NPC	Batalla(Jugador, EntrenadorNPC)	Crea una batalla controlada; esEncuentroSalvaje = false. El NPC usa IA de tipo-ventaja.
Constructor Salvaje	Batalla(Jugador, Criatura)	Crea un encuentro con criatura salvaje; esEncuentroSalvaje = true. Permite lanzar esferas y huir.

4.3 Clase SesionJuego

Paquete: juego.sesion

Singleton de facto que mantiene todo el estado mutable de una partida. Es la única clase que conoce la secuencia completa de NPCs, la zona actual y el estado de la batalla activa.

Método	Firma	Descripción
iniciar()	public void iniciar(String nombre, int starterIdx)	Inicializa el jugador con el starter elegido y construye la lista de 6

Método	Firma	Descripción
		NPCs en orden de dificultad creciente.
explorar()	public Batalla explorar()	40% de probabilidad de encontrar una criatura salvaje proporcional a la zona actual. Retorna la Batalla creada o null.
retarNPC()	public Batalla retarNPC()	Inicia batalla contra el siguiente NPC no derrotado en la secuencia.
cerrarBatalla()	public void cerrarBatalla()	Tras el fin de batalla: avanza el índice NPC, desbloquea zona 2 tras Líder Marina y actualiza la fase a JUEGO_TERMINADO si se derrotaron todos.
guardarSesion()	public void guardarSesion(int id)	Persiste el estado actual en la tabla sesion_juego de PostgreSQL con UPSERT.
cargarSesion()	public void cargarSesion(int id)	Recupera sesión, jugador, criaturas, objetos y esferas desde la base de datos y reconstruye el estado en memoria.

4.4 Clase Movimiento

Paquete: juego.movimientos

Representa un movimiento de batalla con nombre, tipo elemental, categoría (Físico/Especial), poder, precisión y puntos de poder (PP). Expone factories estáticos para los 12 movimientos predefinidos.

Método	Firma	Descripción
usar()	public boolean usar()	Consume 1 PP. Retorna false si ppActual <= 0 (el movimiento no puede usarse).
restaurarPP()	public void restaurarPP()	Restablece ppActual = ppMax. Llamado al final de cada batalla o al curar en el Centro.
tienePP()	public boolean tienePP()	Retorna ppActual > 0. Usado por el controlador antes de permitir un ataque.
Factories (x12)	public static Movimiento asCua()	Ejemplo: asCua() = TipoElemento.FUEGO, poder 40, 25



Método	Firma	Descripción
		PP. Evitan que la llamada deba conocer el orden exacto de los argumentos del constructor.

4.5 Clase PokeEsfera (abstract) y subclases

Paquete: juego.esferas

Jerarquía de esferas de captura. La clase base implementa la lógica de cálculo de tasa y lanzamiento; las subclases solo sobrescriben tasaBase (o el método calcularTasaCaptura en MasterEsfera).

Subclase	tasaBase	Comportamiento especial
EsferaNormal	1.0	Tasa estándar. La más común; el jugador comienza con 5.
EsferaHP	1.5	Más efectiva cuando la criatura tiene menos del 50% de HP.
SuperEsfera	1.5	Tasa fija aumentada, sin condición adicional.
UltraEsfera	2.0	Alta efectividad general.
EsferaTipo	2.5	Bonus adicional si el tipo de la criatura coincide con el tipo configurado en la esfera.
MasterEsfera	999	Sobrescribe lanzar() para retornar true siempre. Captura garantizada.

4.6 Enum TipoElemento

Paquete: juego.enums

Define los 7 tipos elementales y encapsula la tabla de efectividades como el método `getMultiplicador(atacante)`. Al llamar `defensor.getTipo().getMultiplicador(movimiento.getTipo())` se obtiene el multiplicador correcto sin necesidad de matrices externas.

Defensor \ Atacante	FUEGO	AGUA	PLANTA	ELECTRICO	TIERRA	HIELO
FUEGO	0.5×	2×	0.5×	1×	1×	1×
AGUA	0.5×	0.5×	2×	2×	1×	1×
PLANTA	2×	0.5×	0.5×	1×	1×	2×
ELECTRICO	1×	1×	1×	0.5×	0×	1×
TIERRA	0.5×	2×	2×	0.5×	1×	2×
HIELO	2×	1×	1×	1×	1×	0.5×
NORMAL	1×	1×	1×	1×	1×	1×

4.7 Controladores REST (BattleController / GameController)

Paquete: juego.api | Framework: Javalin 6 | Puerto: 7070

Método	Ruta	Clase	Descripción
POST	/api/game/new	GameController	Inicializa sesión. Body: {nombre, starterIndex}. Retorna GameState completo.
GET	/api/game/state	GameController	Retorna el estado actual de la sesión (fase, jugador, NPCs, batalla).
POST	/api/game/explore	GameController	Intenta encuentro salvaje (40%). Retorna GameState; mensaje si no hay encuentro.
POST	/api/game/heal	GameController	Cura el equipo si puedesCurar = true. Solo una vez por NPC derrotado.
POST	/api/game/challenge-npc	GameController	Inicia batalla contra el siguiente NPC en la secuencia.
GET	/api/battle/state	BattleController	Retorna estado de la batalla activa. 404 si no hay batalla.
POST	/api/battle/action	BattleController	Ejecuta acción de turno. Body: {tipo: ATACAR OBJETO ESFERA HUIR, index}. Retorna GameState + log.



4.8 Estructura del Proyecto

```
Proyecto-POO/
├── backend/src/main/java/juego/
│   ├── Main.java # Punto de entrada; arranca Javalin en puerto
│   └── 7070
│       ├── DatabaseConnection.java # Singleton de conexión a PostgreSQL
│       ├── api/
│       │   ├── GameController.java # Rutas de mundo (/api/game/*)
│       │   ├── BattleController.java # Rutas de batalla (/api/battle/*)
│       │   └── DtoMapper.java # Convierte modelo de dominio a JSON-safe DTOs
│       ├── batalla/
│       │   └── Batalla.java # Motor de combate por turnos
│       ├── criaturas/
│       │   ├── Criatura.java # Clase base abstracta
│       │   ├── Capturable.java # Interfaz: detTasaCaptura()
│       │   └── Nivelable.java # Interfaz: getNivel(), getHpActual(),
│       └── getHpMax()
│           ├── CriaturaFuego/Agua/Planta/Electrico/Normal.java
│           └── instancias/ # Ignis, Aqua, Herba, Voltix, Fumeo, Glacius,
│               Terron, Gorgon
│                   ├── entrenadores/
│                   │   ├── Entrenador.java # Clase base abstracta de entrenadores
│                   │   ├── Jugador.java # Jugador humano con bolsa y esferas
│                   │   └── EntrenadorNPC.java # CPU con IA de tipo-ventaja + factories
│                   ├── enums/
│                   │   ├── TipoElemento.java # 7 tipos + tabla de efectividades
│                   │   ├── Categoria.java # FISICO, ESPECIAL
│                   │   └── NivelDificultad.java # FACIL, NORMAL, DIFICIL (+ flag IA activa)
│                   ├── esferas/
│                   │   └── PokeEsfera + 6 subclases
│                   ├── movimientos/
│                   │   └── Movimiento.java # Movimiento + 12 factories estáticos
│                   ├── objetos/
│                   │   ├── Objeto.java # Clase base de objetos de mochila
│                   │   └── ObjetoCuracion.java # Poción / SuperPoción
│                   └── sesion/
│                       └── SesionJuego.java # Estado global de la partida + persistencia
├── BD
│   ├── database/
│   │   ├── Dockerfile # Imagen PostgreSQL con init.sql
│   │   └── init.sql # DDL de tablas: jugador, criatura,
│   └── sesion_juego, etc.
├── ui/src/
│   ├── App.tsx # Enrutador principal
│   ├── api/gameApi.ts # Cliente REST hacia el backend Java
│   ├── components/
│   │   ├── TitleScreen.tsx # Pantalla de inicio / selección de starter
│   │   └── WorldMap.tsx # Mapa con acciones de explorar / retar /
├── curar
├── log)
└── └── BattleScreen.tsx # Interfaz de batalla (HP bars, movimientos,
    └── HPBar.tsx # Barra de HP animada
```



ITESO Universidad
Jesuita de Guadalajara

```
| | | MoveMenu.tsx
| | | MessageBox.tsx
| | | types/game.ts
| | | pom.xml
| | PostgreSQL JDBC
```

```
# Selección de movimiento en batalla
# Cuadro de diálogo/log de eventos
# Interfaces TypeScript del estado del juego
# Maven: dependencias Javalin, Gson,
```




5. Conceptos de POO Aplicados

Concepto POO	Aplicación en el proyecto
Herencia	Criatura → CriaturaFuego → Ignis, Fumeo. Entrenador → Jugador / EntrenadorNPC. PokeEsfera → EsferaNormal, MasterEsfera, etc.
Abstracción	Criatura y Entrenador son clases abstractas; no pueden instanciarse directamente. Criatura#habilidadEspecial() es abstracto y fuerza a cada subtipo a definir su pasiva.
Encapsulamiento	Todos los atributos de Criatura, Jugador y Batalla son privados/protegidos. Solo se exponen mediante getters necesarios; la lógica de cambio de estado está dentro de la clase.
Polimorfismo	Cualquier List<Criatura> puede contener Ignis, Aqua, Terron, etc. Batalla llama a habilidadEspecial() sin saber qué subtipo está usando.
Interfaces	Capturable define detTasaCaptura(). Nivelable define getNivel(), getHpActual(), getHpMax(). PokeEsfera trabaja con Capturable, no con Criatura concreta.
Patrón Factory	Movimiento.ascua(), Movimiento.rayo(), etc. EntrenadorNPC.rivalFacil(), liderMarina(), campeonRex() construyen NPC completos con un solo método.
Composición	Batalla tiene-un Jugador y tiene-un EntrenadorNPC (no extiende ninguno). SesionJuego tiene-una Batalla y tiene-un Jugador.
Enum con comportamiento	TipoElemento encapsula getMultiplicador() internamente; NivelDificultad encapsula getIAActiva().

6. Proyecto funcional

En el siguiente repositorio de Github se pueden encontrar todos los archivos, códigos y documentación necesarios para el correcto funcionamiento del proyecto.

<https://github.com/Haya543/Proyecto-POO-.git>